

Technical Synthesis of Spectral Dynamic Resonance Suppression: Algorithmic Architecture of Oeksound Soothe2 and Implementation Strategies within the Rust-NIH-Plug Ecosystem

The evolution of digital audio signal processing has transitioned from static filtering techniques toward adaptive, data-driven architectures that respond to the idiosyncratic spectral characteristics of complex audio signals. Oeksound Soothe2 stands as a definitive implementation of this evolution, categorized not as a traditional equalizer or a multiband compressor, but as a dynamic resonance suppressor.¹ This technology addresses the fundamental limitations of static equalization, where fixed notches often attenuate desired harmonic content alongside problematic resonances, and multiband compression, which typically lacks the frequency resolution required to target narrow-band harshness without introducing phase distortion at crossover points.³ The technical objective of Soothe2 is the real-time identification and automatic reduction of localized energy buildups, effectively functioning as an adaptive, thousands-of-bands dynamic processor.³ Implementing such a system requires a high-performance framework capable of low-latency spectral manipulation. The Rust-based NIH-plug framework, coupled with the egui interface library, provides a contemporary ecosystem for developing these sophisticated processors, leveraging the safety and concurrency guarantees of the Rust language alongside SIMD-accelerated Fast Fourier Transform (FFT) libraries.⁶

Algorithmic Principles of Dynamic Resonance Suppression

The operational core of Oeksound Soothe2 is a spectral analysis engine that utilizes the Fast Fourier Transform to decompose time-domain audio into a frequency-domain representation.⁴ Unlike time-domain dynamic equalizers that trigger IIR (Infinite Impulse Response) filters based on amplitude thresholds within specific bands, Soothe2 operates bin-by-bin in the frequency domain.⁹ This allows the algorithm to act as a "spectral thresholding" processor, where the magnitude of each frequency bin is evaluated against its neighbors to identify peaks that exceed a dynamically calculated spectral envelope.⁵

The Spectral Analysis Pipeline and Windowing

At the foundation of this process lies the Discrete Fourier Transform (DFT), typically

implemented via the FFT for computational efficiency.⁹ To process a continuous stream of audio, the system employs the Short-Time Fourier Transform (STFT), where the signal is divided into overlapping blocks or "frames".⁹ The choice of block size is a critical design trade-off between frequency resolution and temporal accuracy; larger blocks provide finer frequency bins but smear transients in time.⁹

FFT Parameter	Technical Specification in Suppression Context	Perceptual and Computational Impact
Window Size (N)	512 to 16384 samples (Commonly 4096) ⁹	Determines the minimum resolvable frequency gap (f_s/N). ¹³
Window Function	Hann, Hamming, or Vorbis windows ⁹	Mitigates spectral leakage and boundary discontinuities. ⁹
Overlap Ratio	4x (default), 8x (High), 16x (Ultra) ⁹	Enhances temporal refresh rate and avoids "blocky" artifacts. ⁹
Latency	$\approx$ samples (System dependent) ¹⁵	Compensated by DAW; higher resolution increases delay. ¹⁵

The necessity of windowing arises from the fact that the FFT assumes the signal segment is periodic.⁹ Any discontinuity at the edges of the block would manifest as "spectral leakage," generating spurious high-frequency artifacts across the bins.⁹ By applying a Hann window, which fades the beginning and end of each block to zero, the signal becomes "sine-like" within the transform window, concentrating energy into the correct bins and ensuring a transparent reconstruction when using the overlap-add (OLA) method.⁹

Resonance Detection Logic and Spectral Weighting

The intelligence of a resonance suppressor resides in its detection algorithm, which must distinguish between harmonic partials and unpleasant resonances.¹ In Soothe2, the detection engine identifies problematic areas by comparing the magnitude of an individual bin (or a small group of bins) against the surrounding frequency area.⁵ This is often achieved by generating a threshold curve that represents the general shape of the spectrum, sometimes using standard deviation or moving averages across one-third octave bands or Equivalent Rectangular Bandwidth (ERB) scales.¹⁸

A resonance is flagged when the energy in a specific bin significantly exceeds this threshold

curve.¹⁸ In high-fidelity implementations, the threshold is not purely statistical; it is biased by perceptual models. Human hearing is notably more sensitive to the 2 kHz to 5 kHz region—the area where sibilance and harshness typically reside—meaning the algorithm must be more reactive in these ranges than in the sub-bass or extreme high frequencies.²

Parameter Mapping and Control Architecture

Soothe2 translates its internal spectral math into parameters that mimic the workflow of a compressor or equalizer, though their underlying mechanisms are different.² These parameters govern the sensitivity and character of the generated suppression notches.⁹

Sensitivity and Resolution Controls

The primary user controls in Soothe2—Depth, Sharpness, and Selectivity—form a multi-layered filter for the detection engine.⁹

- **Depth:** Functioning as a global threshold or sensitivity control, Depth determines the minimum level at which the processor begins to attenuate detected resonances.⁹ High depth settings result in more aggressive reduction, potentially reaching notches as deep as 60 dB.¹⁶
- **Sharpness:** This parameter controls the width or "Q" of the generated suppression filters.⁹ Lower sharpness results in broad, smooth tonal shaping, while higher sharpness targets narrow, surgical frequencies such as the whistling in a vocal or the ringing of a drum.¹⁶
- **Selectivity:** Selectivity dictates how "picky" the algorithm is regarding the peaks it identifies.⁹ A high selectivity setting ignores subtle fluctuations and focuses only on the most egregious resonances, whereas a low setting allows the processor to affect more of the spectral content, moving it closer to a traditional spectral balancer.¹⁹

Parameter	Analogue in Traditional DSP	Spectral Function
Depth	Threshold / Ratio	Governs the global sensitivity of the detection engine. ⁹
Sharpness	Filter Q-factor	Controls the bandwidth of the per-bin attenuation notches. ⁹
Selectivity	Detection Threshold Bias	Filters out peaks that do not significantly deviate from the average. ²⁰

Soft/Hard Mode	Knee / Algorithm Topology	Toggles between transparent balancing and aggressive correction. ¹
----------------	---------------------------	---

Temporal Dynamics: Attack and Release

Crucially, Soothe2 applies temporal dynamics to each spectral bin.² The Attack parameter determines how quickly a detected resonance is suppressed, while the Release parameter controls how quickly the suppression notch disappears once the energy buildup subsides.²⁰ Because the processing happens in the spectral domain, these timing constants are often frequency-dependent; lower frequencies require longer integration times to avoid "musical noise" and distortion artifacts.¹⁶

Technical Implementation with the NIH-Plug Framework

To develop a plugin with the capabilities of Soothe2, the NIH-plug framework offers a robust, API-agnostic architecture written in Rust.⁶ Rust's inherent memory safety and zero-cost abstractions are ideally suited for the high-throughput, real-time requirements of spectral processing.⁸ NIH-plug supports VST3 and CLAP formats, providing the necessary hooks for audio buffer management, parameter automation, and state persistence.⁶

Framework Core and Buffer Processing

A plugin built in NIH-plug implements the Plugin trait, where the process function serves as the entry point for audio data.²⁴ For spectral processing, the input must be buffered into blocks that match the FFT window size.²⁴ The Buffer struct in NIH-plug provides the iter_blocks method, which allows the developer to group samples into consistent chunks, a prerequisite for windowed FFT analysis.²⁴

Rust

```
// Conceptual structure of a spectral process loop in NIH-plug
fn process(&mut self, buffer: &mut Buffer,...) -> ProcessStatus {
    // Iterate over the buffer in blocks matching our FFT window
    for block in buffer.iter_blocks(self.fft_size) {
        let mut channels = block.into_iter();
        while let (Some(l), Some(r)) = (channels.next(), channels.next()) {
            // 1. Buffer incoming samples into a circular buffer
            // 2. Apply windowing (e.g., Hann)
            // 3. Perform Forward FFT using rustfft
        }
    }
}
```

```

        // 4. Execute detection and suppression logic
        // 5. Perform Inverse FFT
        // 6. Overlap-Add back into the output stream
    }
}
ProcessStatus::Normal
}

```

The use of `rustfft` provides a high-performance, pure-Rust implementation of the FFT.⁷ On `x86_64` architectures, `rustfft` automatically leverages AVX and FMA instructions for hardware acceleration, which is critical when processing thousands of frequency bins in real-time.⁷ The "Spectral Compressor" example within the NIH-plug repository provides a template for this type of processing, demonstrating how to manage up to 16,384 bands while applying upward and downward compression logic.⁶

Parameter Management and Persistence

The NIH-plug framework manages parameters through a dedicated struct decorated with procedural macros.²⁸ For a resonance suppressor, these parameters must be smoothed to prevent zipper noise when the user adjusts the Depth or Sharpness.²⁴ NIH-plug includes built-in parameter smoothing that can operate on an entire block of audio, improving efficiency.²⁴ Furthermore, the `#[persist = "key"]` attribute allows for the seamless saving and restoration of the plugin's state, including UI-related data such as the size and position of the frequency graph.⁶

UI Design and Visualization with `egui`

The user interface of a resonance suppressor is not merely aesthetic; it is a functional diagnostic tool that visualizes the problematic resonances and the intensity of the suppression.¹⁹ NIH-plug integrates with `egui`, an immediate-mode GUI library that provides a high-performance, portable interface for Rust applications.²³

Immediate Mode Visualization Principles

`egui` operates by redrawing the entire interface every frame, typically synchronized with the monitor's refresh rate.²³ This architecture is particularly advantageous for audio plugins because it eliminates the need for complex state synchronization between the UI components and the underlying DSP parameters; the UI simply queries the current state of the parameters and the spectral buffers during each render pass.²³

For spectral visualization, the `egui_plot` crate or custom painters can be used to render the frequency magnitude data.³⁰ This requires the audio thread to periodically send a "snapshot" of the current FFT magnitudes to the UI thread.²⁸

Visualization Element	Implementation Strategy	UX Utility
Real-time Spectrogram	Plotting FFT magnitude bins on a log frequency axis. ³³	Identifies resonant peaks visually. ¹⁹
Suppression Nodes	Draggable egui handles mapped to frequency/sensitivity. ¹⁷	Focuses processing on specific ranges. ¹⁶
Gain Reduction Curve	An inverted curve showing the per-bin attenuation. ¹⁹	Displays where and how much the signal is being cut. ¹
Delta Monitoring	Toggling the audio output to hear only the suppressed content. ¹⁶	Allows auditing of the specific resonances removed. ¹⁵

Designing the Interaction Surface

In Soothe2, the interactive frequency graph uses nodes that work contrary to traditional EQ: moving a node upward increases the sensitivity of suppression in that band, rather than increasing the gain.² This behavior is a "weighting curve" that biases the detection threshold.¹⁷ Implementing this in egui involves mapping mouse coordinates to logarithmic frequency values and using the vertical position to scale the "Selectivity" or "Depth" for specific FFT bins.¹⁷

Data Synchronization and Thread Safety

One of the most complex aspects of implementing a spectral processor with a visualizer is the safe transfer of data between the real-time audio thread and the GUI thread.³⁷ The audio thread must never block, yet the GUI thread requires a consistent stream of high-density spectral data to provide a smooth visual experience.²³

Atomic Triple Buffering and Lock-Free Communication

To bridge these threads, developers often employ atomic triple buffering or lock-free ring buffers.³² In a triple-buffering scheme, three buffers are maintained: one for writing (Audio thread), one for reading (GUI thread), and one "pending" buffer.³²

1. The audio thread completes a spectral analysis frame and writes the magnitudes into the current write buffer.
2. An atomic swap is performed between the write buffer and the pending buffer.³²
3. When the GUI thread is ready to render a new frame, it checks the pending buffer and swaps it with the current read buffer.⁴⁰

This ensures that the audio thread never competes for the same memory as the GUI,

maintaining real-time safety while providing the latest possible data for visualization.³² In NIH-plug, this data transfer can be implemented using Arc and AtomicPtr, or through specialized lock-free queue crates like crossbeam-channel or ringbuffer.²³

Latency Reporting and DAW Synchronization

Because spectral processing requires buffering samples into blocks, it introduces a constant latency ($L = N - \text{overlap}$).⁹ NIH-plug requires the developer to report this latency to the host DAW during the plugin initialization or when the FFT window size is changed.¹⁵ This allows the DAW to apply Delay Compensation, ensuring the processed audio remains in sync with the rest of the project.¹

Optimizing Spectral Performance in Rust

The computational load of a 16,384-band spectral processor is significant.¹² To achieve the efficiency of a commercial plugin like Soothe2, several optimization strategies must be applied within the Rust environment.

SIMD and Hardware Acceleration

Rust's standard library and crates like rustfft provide access to SIMD (Single Instruction, Multiple Data) intrinsics.⁷ By processing multiple frequency bins simultaneously—for example, using AVX-512 or AVX2 instructions—the plugin can reduce the per-frame CPU time significantly.⁷

Oversampling and Anti-Aliasing

To avoid aliasing when using high sharpness settings (which create very narrow, steep notches), internal oversampling is often employed.⁹ By doubling the internal sample rate, the Nyquist frequency is shifted upward, providing more "headroom" for the generated suppression filters to operate without wrapping around the spectrum.⁹ NIH-plug supports linear-phase oversampling through ports like Soft Vacuum, providing a blueprint for high-fidelity internal resampling.⁶

Memory Management and Pre-Allocation

A cardinal rule of real-time audio is to never allocate memory on the audio thread.²³ In Rust, this means all FFT buffers, windowing arrays, and magnitude vectors must be pre-allocated during the initialize phase of the NIH-plug lifecycle.²⁴ Any changes to the FFT size or resolution must trigger a re-initialization rather than a dynamic allocation during the process loop.²⁴

Advanced Logic: Selectivity and Mode Topology

The distinction between Soothe2's "Soft" and "Hard" modes reveals deeper architectural choices in resonance suppression.¹

Algorithmic Modes

Soft mode is generally less level-dependent, acting more as an adaptive spectral balancer.¹⁶ Hard mode increases the compression ratio for the detected peaks and introduces a more aggressive thresholding behavior, making it more effective for extreme problem-solving at the cost of potential transparency.¹

Feature	Soft Mode Implementation	Hard Mode Implementation
Suppression Ratio	Lower, more gradual attenuation. ¹⁶	Higher, "notching" style attenuation. ¹⁶
Detection Sensitivity	Broadly focused on the spectral average. ¹⁶	Focused on high-deviation transients. ¹⁶
Transient Handling	Preserves the initial impact of the signal. ¹	Suppresses resonances even within transients. ²²

The Role of Median vs. Mean Filtering

In the detection phase, the algorithm must calculate a "baseline" spectrum to determine what constitutes a resonance.¹⁸ Research indicates that median filtering is often superior to mean (average) filtering for peak detection in noisy signals.⁴² While a mean filter smears peaks, a median filter preserves sharp edges and is more robust against outliers, allowing the detection engine to more accurately isolate a narrow resonance from the surrounding harmonic content.⁴²

Building the Sidechain Integration

Sidechain processing is a defining feature of modern spectral tools.¹ In this configuration, the plugin accepts a secondary audio input (the sidechain) and uses its spectral magnitude to trigger suppression on the primary input.¹

1. **Dual FFT Analysis:** The plugin performs two simultaneous forward FFTs: one on the main input and one on the sidechain.¹
2. **Magnitude Mapping:** The sidechain's magnitude spectrum is used to define the dynamic threshold for the main input.¹
3. **Spectral Carving:** Where the sidechain has high energy (e.g., a vocal), the main input is suppressed (e.g., a guitar bus), effectively carving out a frequency-dependent "pocket" for the sidechain signal to inhabit.³

This implementation requires NIH-plug to be configured with auxiliary inputs, a feature fully supported by the CLAP and VST3 standards.⁶

Conclusion and Implementation Roadmap

Synthesizing Oeksound Soothe2's functionality within the Rust/NIH-plugin ecosystem requires a convergence of high-performance DSP, thread-safe data management, and intuitive UI design. The core mechanism—dynamic resonance suppression through bin-by-bin spectral thresholding—relies on the STFT architecture, where windowing and overlap-add ensure a transparent reconstruction of the audio signal.⁹ By mapping compression-style parameters (Attack, Release, Threshold/Depth) to individual frequency bins, the processor achieves a level of precision that is fundamentally unattainable with traditional time-domain filters.⁴ For the developer, NIH-plugin provides the essential scaffolding to manage the complexity of multi-threaded audio processing, while egui offers the flexibility to build a highly visual and interactive control surface.⁶ The path to a professional implementation involves:

- Utilizing rustfft for SIMD-accelerated frequency domain transformations.⁷
- Employing atomic triple buffering to synchronize spectral data with the UI without compromising real-time safety.³²
- Implementing perceptual weighting curves and sophisticated peak detection algorithms, such as median filtering, to ensure the suppression targets only the unpleasant artifacts of a recording.¹⁴
- Configuring oversampling and high-resolution FFT overlaps to minimize artifacts and maximize the surgical accuracy of the suppression.⁹

This technical framework not only allows for the replication of existing technologies like Soothe2 but also provides a foundation for further innovation in adaptive, intelligent audio processing. The Rust language's unique properties, combined with the NIH-plugin framework, represent a significant advancement for the audio plugin industry, enabling the creation of tools that are both safer and more computationally efficient than their predecessors.

Works cited

1. soothe2 - oeksound, accessed on December 22, 2025, <https://oeksound.com/plugins/soothe2/>
2. Soothe 2: The Plugin That Finally Made Sense to Me - Elevar Magazine, accessed on December 22, 2025, <https://www.elevarmagazine.com/industry/soothe-2-the-plugin-that-finally-made-sense-to-me>
3. You're Wrong About Soothe2, Gullfoss, and Bloom : r/audioengineering - Reddit, accessed on December 22, 2025, https://www.reddit.com/r/audioengineering/comments/1esd5xw/youre_wrong_about_soothe2_gullfoss_and_bloom/
4. Resonance Suppressor vs. Multi-band Compressor vs. Dynamic EQ: Which Should You Use? | Baby Audio, accessed on December 22, 2025, <https://babyaud.io/blog/resonance-suppression-vs-multiband-compression>
5. Soothe/Smooth Operator vs Dynamic EQ. - Page 3 - Effects Forum - KVR Audio, accessed on December 22, 2025,

- <https://www.kvraudio.com/forum/viewtopic.php?t=605330&start=30>
6. robbert-vdh/nih-plugin: Rust VST3 and CLAP plugin framework and plugins - because everything is better when you do it yourself - GitHub, accessed on December 22, 2025, <https://github.com/robbert-vdh/nih-plugin>
 7. rustfft - Rust - Docs.rs, accessed on December 22, 2025, <https://docs.rs/rustfft/>
 8. NIH-plugin Is A Plugin Framework With Eight FREE Plugins (Including A Disperser Clone), accessed on December 22, 2025, <https://bedroomproducersblog.com/2023/03/14/nih-plugin/>
 9. Oeksound Soothe Review - Admiral Bumblebee, accessed on December 22, 2025, <https://www.admiralbumblebee.com/music/2018/10/27/Oeksound-Soothe-Review.html>
 10. Is there a plugin that suppresses the frequency below a threshold? - Page 2 - Sound Design Forum - KVR Audio, accessed on December 22, 2025, <https://www.kvraudio.com/forum/viewtopic.php?p=8916153>
 11. Implementation of DSP Techniques for Noise Cancellation in Audio Signals - ijrmeet, accessed on December 22, 2025, https://ijrmeet.org/wp-content/uploads/2018/04/IJRMEET0418050009_Implementation%20of%20DSP%20Techniques%20for%20Noise%20Cancellation%20in%20Audio%20Signals.pdf
 12. Robbert van der Helm Releases FREE Spectral Compressor - Bedroom Producers Blog, accessed on December 22, 2025, <https://bedroomproducersblog.com/2024/03/03/nih-plugin-spectral-compressor/>
 13. Frequency Spectrum Analysis with FFT in Rust (library spectrum-analyzer) - phips blog, accessed on December 22, 2025, <https://phip1611.de/blog/frequency-spectrum-analysis-with-fft-in-rust/>
 14. A Hybrid DSP/Deep Learning Approach to Real-Time Full-Band Speech Enhancement - arXiv, accessed on December 22, 2025, <https://arxiv.org/pdf/1709.08243>
 15. Soothe2 "Clone" - Free M4L Device : r/ableton - Reddit, accessed on December 22, 2025, https://www.reddit.com/r/ableton/comments/15l7zsi/soothe2_clone_free_m4l_device/
 16. soothe2 manual - oeksound, accessed on December 22, 2025, <https://oeksound.com/manuals/soothe2/>
 17. Soothe Live manual - oeksound, accessed on December 22, 2025, <https://oeksound.com/manuals/soothe-live/>
 18. Method for Resonant Frequency Attenuation in Dynamic Audio Equalizer - MDPI, accessed on December 22, 2025, <https://www.mdpi.com/2076-3417/15/6/3038>
 19. oeksound soothe2 €199 | Computer Music August 2020 - Pocketmags, accessed on December 22, 2025, <https://pocketmags.com/eu/computer-music-magazine/august-2020/articles/78222/oeksound-soothe2-199>
 20. How to Use Soothe by Oeksound [Plugin Tutorial] - Mastering.com, accessed on December 22, 2025, <https://mastering.com/soothe/>

21. Adding Quality And Clarity To Your Mix with Oeksound's Soothe2 - Attack Magazine, accessed on December 22, 2025, <https://www.attackmagazine.com/technique/tutorials/adding-clarity-and-quality-with-oeksound-soothe2/>
22. [The Music Telegraph] 'Soothe 2': Oeksound's Second Take on The Concept of Resonance Removals, accessed on December 22, 2025, <https://www.themusictelegraph.com/772>
23. egui: an easy-to-use GUI in pure Rust - Crates.io, accessed on December 22, 2025, <https://crates.io/crates/egui>
24. Buffer in nih_plugin::buffer - Rust, accessed on December 22, 2025, https://nih-plugin.robbertvanderhelm.nl/nih_plugin/buffer/struct.Buffer.html
25. nih_plugin::buffer - Rust, accessed on December 22, 2025, https://nih-plugin.robbertvanderhelm.nl/nih_plugin/buffer/index.html
26. spectrum-analyzer - crates.io: Rust Package Registry, accessed on December 22, 2025, <https://crates.io/crates/spectrum-analyzer>
27. Spectral Compressor FREE - Cakewalk Discuss, accessed on December 22, 2025, <https://discuss.cakewalk.com/topic/75227-spectral-compressor-free/>
28. nih_plugin_egui - Rust, accessed on December 22, 2025, https://nih-plugin.robbertvanderhelm.nl/nih_plugin_egui/index.html
29. egui - Rust - Docs.rs, accessed on December 22, 2025, <https://docs.rs/egui/latest/egui/>
30. emilk/egui_plot: 2D plotting library in Rust for egui - GitHub, accessed on December 22, 2025, https://github.com/emilk/egui_plot
31. Visualization — list of Rust libraries/crates // Lib.rs, accessed on December 22, 2025, <https://lib.rs/visualization>
32. [Synchronisation] How to implement triple buffering - NI Community, accessed on December 22, 2025, <https://forums.ni.com/t5/LabVIEW/Synchronisation-How-to-implement-triple-buffering/td-p/3025787>
33. Spectrum Viewer — Open Ephys GUI Docs - GitHub Pages, accessed on December 22, 2025, <https://open-ephys.github.io/gui-docs/User-Manual/Plugins/Spectrum-Viewer.html>
34. spectrum-analyzer - crates.io: Rust Package Registry, accessed on December 22, 2025, <https://crates.io/crates/spectrum-analyzer/1.2.1>
35. blitzar-tech/egui_graphs: Interactive graph visualization widget for rust powered by egui and petgraph - GitHub, accessed on December 22, 2025, https://github.com/blitzar-tech/egui_graphs
36. Review: Soothe2 By OekSound - Seed To Stage, accessed on December 22, 2025, <https://seedtostage.com/review-soothe-by-oek-sound/>
37. Audio-GUI Thread Processing - Will Pirkle, accessed on December 22, 2025, <http://willpirkle.com/Downloads/Audio-GUI%20Thread%20Processing.pdf>
38. Sync GUI from Audio Thread: any way to copy bulk of data with one instruction? - DSP and Plugin Development Forum, accessed on December 22, 2025, <https://www.kvraudio.com/forum/viewtopic.php?t=498387>

39. Python Cookbook-O'Reilly (2005) | PDF - Scribd, accessed on December 22, 2025, <https://www.scribd.com/document/513292957/Python-Cookbook-O-Reilly-2005>
40. Atomic ring/dual buffer implementation? - Page 2 - DSP and Plugin Development Forum, accessed on December 22, 2025, <https://www.kvraudio.com/forum/viewtopic.php?p=8934807>
41. Rust library: audio-visualizer - Lib.rs, accessed on December 22, 2025, <https://lib.rs/crates/audio-visualizer>
42. Comparison of processing results of median filter and mean filter on Gaussian noise, accessed on December 22, 2025, https://www.researchgate.net/publication/372202005_Comparison_of_processing_results_of_median_filter_and_mean_filter_on_Gaussian_noise
43. Robustness of Median Filter for Suppression of Salt and Pepper Noise (SPN) and Random Valued Impulse Noise (RVIN) - CSC Journals, accessed on December 22, 2025, <https://www.cscjournals.org/manuscript/Journals/IJIP/Volume12/Issue1/IJIP-1143.pdf>
44. Comparison between Mean Filter and Median Filter Algorithm in Image Denoising Field, accessed on December 22, 2025, https://www.researchgate.net/publication/286750387_Comparison_between_Mean_Filter_and_Median_Filter_Algorithm_in_Image_Denoising_Field
45. Performance Evaluation of Preprocessing Techniques for Noise Reduction and Enhancement in MRI-based Brain Tumor Detection - IJSDR, accessed on December 22, 2025, <https://ijsdr.org/papers/IJSDR2502108.pdf>